# SIMATS SCHOOL OF ENGINEERING
# SAVEETHA INSTITUTE OF MEDICAL AND
# TECHNICAL SCIENCES
# CHENNAI-602105

## ASSESSMENT 4

## CSA1006-SOFTWARE ENGINEERING


Submitted By

**Name:** K. Srinivasa Reddy

**Register Number:** 192525452


**PROBLEM 1:**

**Explain technical debt. What are the major causes of technical debt, and how can organizations minimize it during software evolution?**

Introduction

- Technical debt is a concept in software engineering that describes the additional work, effort, and cost that may be required in the future because of choosing a quick, temporary, or less effective solution during software development. Developers may sometimes take shortcuts to meet deadlines, satisfy immediate requirements, or release software quickly. Although these shortcuts can provide short-term benefits, they can create problems that become more expensive to solve as the software continues to evolve.
- Technical debt is similar to financial debt. In financial debt, borrowing money provides an immediate benefit but requires repayment later with additional interest. Similarly, in software development, taking shortcuts may help complete a feature quickly, but the organization may have to spend more time and resources later to fix, improve, test, or maintain the software.

- Technical debt is not always caused by poor developers. It can also occur because of changing requirements, business pressure, limited resources, outdated technologies, or decisions that were reasonable when the software was originally developed.

**Major Causes of Technical Debt 1. Tight Project Deadlines**

- When a project has a strict deadline, developers may choose a quick solution rather than spending additional time designing a better solution. This can result in poorly structured code, incomplete features, or insufficient testing.
- **For example**, a developer may implement a simple solution to complete a feature before a release deadline. Later, when more features are added, the original implementation may become difficult to modify.

**2. Poor Software Design**

- Poor architecture and design decisions can create technical debt. If the system is not properly designed for scalability, maintainability, and future changes, developers may have difficulty adding new functionality.
- A poorly designed application may require developers to modify several unrelated components whenever a small requirement changes.

**3. Lack of Documentation**

- Proper documentation helps developers understand the architecture, functionality, dependencies, and implementation of a system. When documentation is missing or outdated, developers may spend additional time understanding existing code.

- This increases maintenance effort and can result in incorrect modifications.

**4. Insufficient Testing**

- If software is released without sufficient testing, bugs and defects may remain in the system. These defects can become more difficult and expensive to fix later.
- Automated testing, unit testing, integration testing, and system testing help organizations identify problems early.

**5. Outdated Technologies**

- Using outdated programming languages, libraries, frameworks, or development tools can create technical debt. Older technologies may have limited support, security vulnerabilities, compatibility problems, or difficulty integrating with modern systems.

- Regularly updating dependencies and technologies can help reduce this type of debt.

**6. Frequent Requirement Changes**

- Software requirements often change because of customer needs, business decisions, or market conditions. If changes are continuously added without modifying the underlying architecture, the software can become complicated.
- Repeated modifications without proper redesign can result in tightly coupled and difficult-tomaintain code.

**7. Poor Coding Practices**

- Duplicate code, complex logic, poor naming conventions, large functions, and improper organization can make software difficult to understand and maintain.
- Following coding standards and conducting code reviews can help prevent these problems.

**8. Lack of Refactoring**

- Refactoring means improving the internal structure of existing code without changing its external behavior. If developers never refactor old code, complexity can gradually increase.
- Regular refactoring keeps the code clean, understandable, and easier to modify.
- How Organizations Can Minimize Technical Debt

**1. Use Good Software Architecture**

 Organizations should select suitable architectures and design patterns before implementing large systems. A well-designed architecture makes future modifications easier.

**2. Perform Regular Code Reviews**

 Code reviews allow developers to identify poor coding practices, duplicate code, security problems, and design issues before they become major problems.

**3. Perform Regular Refactoring**

 Developers should allocate time for improving existing code. Refactoring can reduce complexity and make future development easier.

**4. Use Automated Testing**

 Automated tests help detect defects whenever changes are made to the software. This reduces the possibility of introducing new problems while modifying existing code.

**5. Maintain Proper Documentation**

 Documentation should explain important architectural decisions, system components, APIs, dependencies, and configuration. It should also be updated when the software changes.

**6. Keep Technologies Updated**

 Organizations should regularly update libraries, frameworks, dependencies, and development tools. This helps reduce compatibility, security, and maintenance problems.

**7. Track Technical Debt**

 Technical debt should be identified and recorded instead of being ignored. Teams can maintain a list of technical debt items and prioritize them based on their impact.

**8. Include Technical Debt in Project Planning**

 Development teams should reserve time in their development cycles for fixing technical debt. Technical improvements should be considered along with new features.

**Conclusion**

- Technical debt is an important issue during software evolution because software continuously changes and grows. Although some technical debt may be unavoidable, uncontrolled technical debt can increase maintenance costs, reduce software quality, and slow down development. Organizations can minimize it through good architecture, coding standards, code reviews, testing, documentation, regular refactoring, and proper planning.

**PROBLEM 2:**
**Explain how AI/ML and Edge Computing are transforming modern software engineering. Provide one practical example for each.**

**Introduction:**

- Modern software engineering is changing rapidly because of technologies such as Artificial Intelligence (AI), Machine Learning (ML), and Edge Computing. These technologies are helping organizations develop software that

  is more intelligent, automated, responsive, and efficient.
- AI and ML can assist developers in different stages of software development, while Edge Computing allows applications to process data closer to where the data is generated.

- AI/ML in Modern Software Engineering
- Artificial Intelligence (AI) refers to technologies that enable computers and software systems to perform tasks that normally require human intelligence.
- Machine Learning (ML) is a branch of AI in which systems learn patterns from data and use those patterns to make predictions or decisions.
- AI and ML are increasingly being used in software engineering to automate repetitive tasks, improve software quality, identify defects, and optimize development processes.
- Applications of AI/ML in Software Engineering

**1. Automated Code Generation**

- AI-based development tools can assist developers by generating code based on naturallanguage requirements or existing code patterns.
- This can reduce the time required to implement common programming tasks.

**2. Bug Detection**

- Machine learning can analyze previous software defects and identify patterns that may indicate potential bugs in new code.
- This allows developers to identify problematic parts of an application earlier.

**3. Automated Software Testing**

- AI can assist in generating test cases, identifying important test scenarios, and prioritizing tests.
- This can improve testing efficiency and reduce manual testing effort.

**4. Predictive Maintenance**

- ML models can analyze historical system data to predict possible failures or performance problems.
- Organizations can take preventive action before major failures occur.

**5. Performance Optimization**

 AI and ML can analyze application performance and resource usage. The results can help developers identify inefficient components and optimize them.

**6. Security Threat Detection**

 AI-based systems can identify unusual behavior and patterns that may indicate security attacks or vulnerabilities.

This can help organizations improve software security.

**Practical Example of AI/ML**

- A practical example is an ML-based software bug prediction system.
- Suppose an organization has data from previous software projects containing information about source-code complexity, previous defects, number of code changes, and testing history. An ML model can learn patterns from this historical data.
- The model can then analyze new software modules and predict which modules are more likely to contain defects.
- Developers can use these predictions to:
- Prioritize testing.
- Review high-risk modules.
- Identify potential problems earlier.
- Reduce debugging time.
- Improve overall software quality.
- Thus, AI/ML can make the software development process more intelligent and data-driven.

- Edge Computing in Modern Software Engineering
  Edge Computing is a computing approach in which data is processed closer to the location where it is generated instead of sending all the data to a centralized cloud server.
- Traditional cloud-based applications often send data from devices to a remote cloud server for processing. This can introduce network delay and require significant bandwidth.
- Edge Computing moves some processing closer to users, sensors, cameras, machines, or other datagenerating devices.

**Advantages of Edge Computing**

**1. Lower Latency**

- Because data is processed closer to the source, applications can respond more quickly.
- This is important for applications that require real-time responses.

**2. Reduced Bandwidth Usage**

Instead of transferring all raw data to the cloud, edge devices can process and filter the data locally.

 Only important information may need to be sent to the cloud.

**3. Faster Decision-Making**

 Local processing allows systems to make decisions quickly without waiting for communication with a distant server.

**4. Improved Reliability**

 Some applications can continue performing important processing even when the connection to the cloud is slow or temporarily unavailable.

**5. Better Privacy**

In some situations, sensitive information can be processed locally rather than sending all raw data to a remote server.

- Practical Example of Edge Computing
- A practical example is a smart traffic management system.
- Traffic cameras and sensors can continuously collect information about vehicles, traffic density, and road conditions. Instead of sending every video frame to a remote cloud server, nearby edge devices can process the information locally.
- The edge device can identify:
- Number of vehicles
- Traffic congestion
- Accidents
- Traffic flow  Vehicle movement
- If heavy traffic is detected, the system can immediately adjust traffic signals.
- Only important or summarized information may then be sent to the cloud for long-term analysis and reporting.
- This reduces network traffic and provides faster responses.
- Conclusion

AI/ML and Edge Computing are transforming modern software engineering in different but complementary ways. AI/ML improves automation, prediction, testing, security, and software quality, while Edge Computing improves application responsiveness by processing data closer to its source. These technologies help organizations build intelligent, efficient, reliable, and responsive software systems.

- **PROBLEM 3:**
  **Case Study – A company deploys dozens of microservices in the cloud and notices that resource usage and energy consumption are increasing. Explain three sustainable DevOps practices that can reduce energy consumption and cloud cost.**

**Introduction**

- In a cloud-based microservices architecture, many services may run simultaneously. If these services are not properly optimized, the company may use more CPU, memory, storage, and network resources than necessary. This increases both cloud costs and energy consumption.
- Sustainable DevOps focuses on improving software development and deployment practices while reducing unnecessary resource consumption.
- Three important practices that can be used in this case are cloud resource optimization, autoscaling and container optimization, and sustainable CI/CD with continuous monitoring.

- 1. Cloud Resource Optimization
- The company should continuously monitor the resources used by its microservices.

**Important resources include:**

- CPU
- Memory
- Storage
- Network bandwidth
- Database resources
- Cloud instances
  Sometimes cloud services are given more resources than they actually need. For example, a microservice may be allocated a large amount of CPU and memory even though it uses only a small percentage of those resources.
- The company can monitor resource usage and resize or remove unnecessary resources.

**How It Reduces Energy Consumption**

 Unused or over-provisioned resources consume computing power unnecessarily. Reducing these resources decreases the amount of processing required by cloud infrastructure.

**How It Reduces Cloud Cost**

 Most cloud providers charge based on resource usage. Therefore, removing unused resources and selecting appropriate resource sizes can reduce the monthly cloud bill.

**Example**

If a microservice requires only 2 GB of memory but is allocated 8 GB continuously, the company can optimize its configuration and allocate resources closer to the actual requirement.

2. Autoscaling and Container Optimization

- The company can use containers and autoscaling to efficiently manage its microservices.
- Autoscaling automatically increases or decreases the number of running service instances according to workload.
- For example, during peak hours, a service may receive thousands of requests. The system can automatically start additional instances to handle the increased workload.
- During low-traffic periods, unnecessary instances can be stopped or scaled down.
- Benefits
- Prevents unnecessary resources from running.
- Improves resource utilization.
- Handles changing workloads efficiently.
- Reduces energy consumption.
- Reduces cloud costs.

Example

- Suppose an online shopping application normally requires five instances during the day but only one or two instances at night.
- Instead of running five instances continuously, autoscaling can reduce the number of instances during low-traffic periods and increase them when demand rises.
- This prevents unnecessary computing and saves resources.
- Container Optimization

- Containers allow multiple microservices to run efficiently on shared infrastructure.
  Properly configuring container resource limits and requests can prevent individual services from consuming excessive resources.
- Efficient container management therefore contributes to sustainable cloud operations.

**3. Sustainable CI/CD and Continuous Monitoring**

- Continuous Integration and Continuous Deployment (CI/CD) pipelines are frequently used in modern DevOps environments.
  If a company has dozens of microservices, inefficient CI/CD pipelines can consume a significant amount of computing resources.
- The organization should optimize its CI/CD processes by avoiding unnecessary builds, tests, and deployments.
- Sustainable CI/CD Practices
- Avoid unnecessary pipeline executions.
- Run only relevant tests when possible.
- Reuse build artifacts.
- Remove unused CI/CD resources.
- Optimize automated testing.
- Monitor pipeline resource consumption.
- Continuous Monitoring
- The company should continuously monitor the performance and resource usage of its microservices.
- Monitoring can identify:
- High CPU usage
- High memory consumption
- Underutilized services
- Unnecessary cloud instances
- Resource-intensive applications
- Inefficient workloads
- The development and operations teams can then take corrective action.  Benefits
  Sustainable CI/CD and monitoring can:
- Reduce unnecessary computing.
- Reduce energy consumption.
- Improve development efficiency.
- Identify resource wastage.
- Reduce cloud costs.
- Improve overall system performance.

**Overall Conclusion**

For a company running dozens of cloud-based microservices, increasing resource usage can result in higher energy consumption and increased cloud costs. Sustainable DevOps practices can help solve this problem.

**The company should:**

- Optimize cloud resources by monitoring and removing unnecessary or over-provisioned resources.
- Use autoscaling and container optimization to run only the resources required for the current workload.

Optimize CI/CD pipelines and continuously monitor systems to identify and eliminate inefficient resource usage.

- These practices allow the company to maintain reliable and scalable microservices while reducing unnecessary energy consumption, resource usage, and cloud expenditure.